Елисеева Е.Р. 090303-ПИа-о24

ЛАБОРАТОРНАЯ РАБОТА №4
Тема: Модули, исключения, файлы и сериализация данных в Python
Вариант №5: Каталог фильмов

1. Цель работы

Получить практические навыки разработки многофайловых Python-приложений, обработки исключительных ситуаций и сохранения состояния программы во внешних файлах.

2. Постановка задачи

Разработать каталог фильмов. Для каждого фильма необходимо хранить: название, жанр, год выпуска, рейтинг. Программа должна быть разделена на отдельные логические модули, поддерживать добавление фильмов, поиск, фильтрацию, сортировку, вычисление статистики, автоматическое сохранение и загрузку структуры данных из JSON, экспорт в CSV-таблицу, а также выбрасывать кастомное исключение при вводе некорректного рейтинга.

3. Краткие теоретические сведения

Модулем в языке Python называется любой файл с расширением .py. Разбиение программы на модули позволяет структурировать проект, отделяя интерфейс пользователя от бизнес-логики и работы с файлами. Для сохранения сложных объектов классов используется сериализация — перевод данных в обычные словари с последующим сохранением через стандартный модуль json. Обратное восстановление объектов из файлов (десериализация) реализуется методами класса с использованием декоратора @classmethod.

Работа с файлами организуется через контекстный менеджер with open(), гарантирующий закрытие файлов даже при возникновении непредвиденных сбоев. Для защиты данных от неконтролируемого затирания или записи неверных значений применяется инкапсуляция с использованием свойств @property и сеттеров, генерирующих исключения.

4. Исходный код программы

Файл: exceptions.py

```
class InvalidRatingError(Exception):
    pass
```

Файл: models.py

```
class Movie:
    def __init__(self, name, genre, year, rating):
        self.name = name
        self.genre = genre
        self.year = year
        self.rating = rating

    @property
    def rating(self):
        return self._rating

    @rating.setter
    def rating(self, value):
        from exceptions import InvalidRatingError
        if value < 0 or value > 10:
            raise InvalidRatingError("рейтинг должен быть строго
от 0 до 10!")
        self._rating = value

    def to_dict(self):
        return {
            "name": self.name,
            "genre": self.genre,
            "year": self.year,
            "rating": self.rating
        }

    @classmethod
    def from_dict(cls, data):
        return cls(
            name=data["name"],
            genre=data["genre"],
            year=data["year"],
            rating=data["rating"]
        )

    def __str__(self):
```

```
        return f"'{self.name}' | Жанр: {self.genre} | Год:
{self.year} | Рейтинг: {self.rating}"
```

Файл: storage.py

```
import json
import csv
import os
from models import Movie

def load_movies_from_json():
    if not os.path.exists("data.json"):
        return []

    try:
        with open("data.json", "r", encoding="utf-8") as file:
            dict_list = json.load(file)
            return [Movie.from_dict(item) for item in dict_list]
    except Exception:
        print("ошибка при чтении файла JSON, создан пустой
список.")
        return []

def save_movies_to_json(movies_list):
    dict_list = [m.to_dict() for m in movies_list]
    with open("data.json", "w", encoding="utf-8") as file:
        json.dump(dict_list, file, ensure_ascii=False, indent=4)

def export_to_csv(movies_list):
    with open("data.csv", "w", newline="", encoding="utf-8") as
file:
        writer = csv.writer(file)
        writer.writerow(["Название", "Жанр", "Год", "Рейтинг"])
        for m in movies_list:
            writer.writerow([m.name, m.genre, m.year, m.rating])
```

Файл: services.py

```
def search_by_genre(movies, search_genre):
    return [m for m in movies if m.genre.lower() ==
search_genre.lower()]

def filter_by_rating(movies, min_rating):
    return [m for m in movies if m.rating >= min_rating]

def sort_movies_by_rating(movies):
    return sorted(movies, key=lambda x: x.rating, reverse=True)

def calculate_statistics(movies):
    if not movies:
        return 0, None
    all_ratings = [m.rating for m in movies]
    avg_rating = round(sum(all_ratings) / len(movies), 2)
    max_movie = max(movies, key=lambda x: x.rating)
    return avg_rating, max_movie
```

Файл: main.py

```
from exceptions import InvalidRatingError
import services
```

```
from storage import load_movies_from_json, save_movies_to_json,
export_to_csv

def get_num_input(prompt, is_float=False):
    while True:
        try:
            return float(input(prompt)) if is_float else
int(input(prompt))
        except ValueError:
            print("ошибка. введите числовое значение!")

def main():
    movies = load_movies_from_json()

    while True:
        print("\n===== МЕНЮ КАТАЛОГА ФИЛЬМОВ (ЛАБА 4) =====")
        print("1. добавить | 2. показать все | 3. поиск по жанру
| 4. фильтр | 5. сорт | 6. стат | 7. CSV | 8. удалить | 0.
выход")
        choice = input("выберите действие: ").strip()

        if choice == "1":
            name_i = input("введите название фильма: ").strip()
            genre_i = input("введите жанр фильма: ").strip()

            while True:
                try:
                    year_i = int(input("введите год выпуска: "))
                    if year_i < 1895 or year_i > 2026:
                        print("в этом году нет фильмов :
( попробуйте другой:")
                        continue
                    break
                except ValueError:
                    print("введите год целым числом!")

            while True:
                try:
                    rating_i = float(input("введите рейтинг
(0-10): "))
                    from models import Movie
                    new_movie = Movie(name_i, genre_i, year_i,
rating_i)
                    movies.append(new_movie)
                    save_movies_to_json(movies)
                    print(f"фильм '{name_i}' успешно добавлен и
сохранен!")
                    break
                except InvalidRatingError as e:
                    print(f"ошибка (сработало исключение): {e}")
                except ValueError:
                    print("введите число!")

        elif choice == "2":
            if not movies:
                print("каталог фильмов пока пуст :(")
                continue
            print("\nсписок всех фильмов:")
```

```
            for idx, m in enumerate(movies, 1):
                print(f"{idx}. {m}")

        elif choice == "3":
            g = input("введите жанр для поиска: ").strip()
            found = services.search_by_genre(movies, g)
            if found:
                print(f"\nнайденные фильмы в жанре '{g}':")
                for m in found:
                    print(f"- {m}")
            else:
                print(f"фильмы в жанре '{g}' не найдены")

        elif choice == "4":
            r = get_num_input("показать фильмы с рейтингом не
ниже: ", is_float=True)
            filtered = services.filter_by_rating(movies, r)
            if filtered:
                print(f"\nфильмы с рейтингом >= {r}:")
                for m in filtered:
                    print(f"- {m}")
            else:
                print("подходящие фильмы отсутствуют")

        elif choice == "5":
            if not movies:
                print("каталог пуст.")
                continue
            sorted_list = services.sort_movies_by_rating(movies)
            print("\nфильмы, отсортированные по убыванию
рейтинга:")
            for m in sorted_list:
                print(f"- {m}")

        elif choice == "6":
            avg_r, max_m = services.calculate_statistics(movies)
            if not max_m:
                print("нет данных для статистики")
                continue
            print(f"\nсредний рейтинг всех фильмов: {avg_r}")
            print(f"фильм с максимальным рейтингом: {max_m}")

        elif choice == "7":
            if not movies:
                print("нечего экспортировать, добавьте фильмы!")
                continue
            export_to_csv(movies)
            print("данные успешно экспортированы в файл
data.csv!")

        elif choice == "8":
            if not movies:
                print("список пуст, удалять нечего.")
                continue
            idx = get_num_input("введите номер фильма для
удаления: ") - 1
            if 0 <= idx < len(movies):
                removed = movies.pop(idx)
```

```
                    save_movies_to_json(movies)
                    print(f"фильм '{removed.name}' успешно удален!")
                else:
                    print("неверный номер!")

            elif choice == "0":
                print("работа программы завершена")
                break
            else:
                print("неверный пункт меню, выберите от 0 до 8")

if __name__ == "__main__":
    main()
```

5. Результаты тестирования программы

Тест 1. Первый запуск программы при отсутствии файла данных (data.json) показан на рисунке 4.1. Проверка корректной работы условия отсутствия файла.

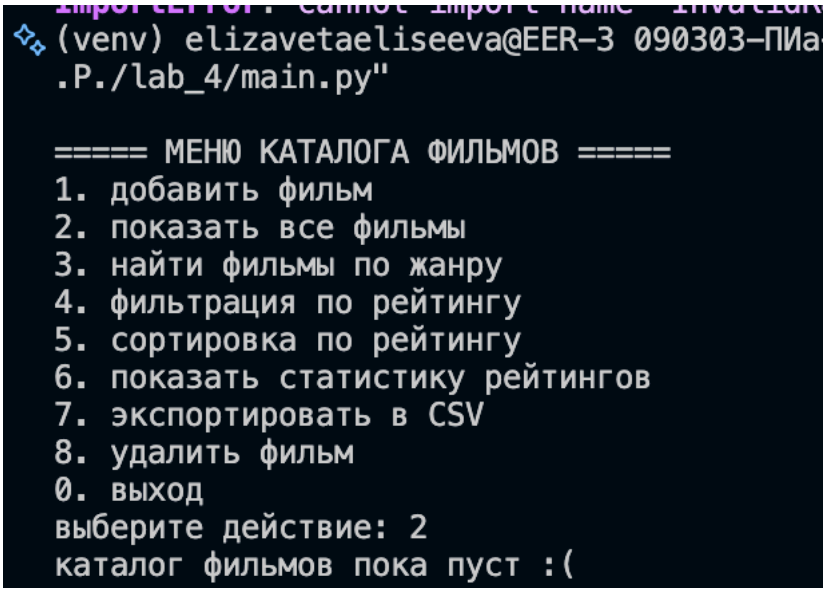


Рисунок 4.1 - пустой список фильмов

Тест 2. Добавление нескольких корректных объектов фильмов и автоматическое создание/сохранение структуры в файл data.json. На рисунке 4.2 приведено добавление нового фильма, а на скриншоте 4.3 - файл data.json.

```
===== МЕНЮ КАТАЛОГА ФИЛЬМОВ =====
1. добавить фильм
2. показать все фильмы
3. найти фильмы по жанру
4. фильтрация по рейтингу
5. сортировка по рейтингу
6. показать статистику рейтингов
7. экспортировать в CSV
8. удалить фильм
0. выход
выберите действие: 1
введите название фильма: интерстеллар
введите жанр фильма: фантастика
введите год выпуска: 2014
введите рейтинг (0–10): 8.6
фильм 'интерстеллар' успешно добавлен и сохранен!
```

Рисунок 4.2 - добавление нового фильма

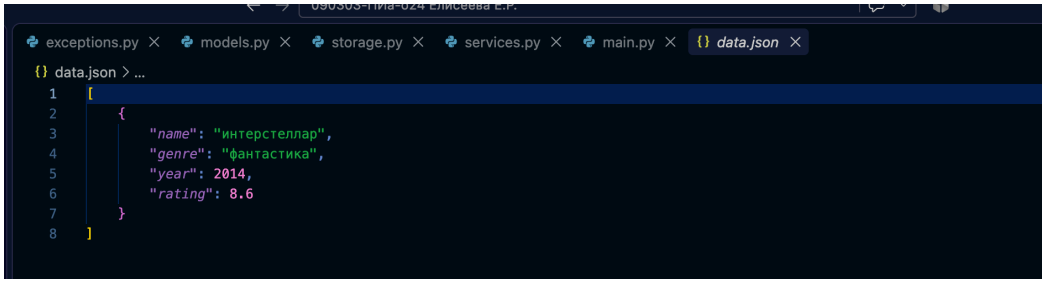


Рисунок 4.3 - файл data.json

Тест 3. Повторный запуск приложения и проверка автоматического считывания и десериализации сохраненных данных с помощью @classmethod.

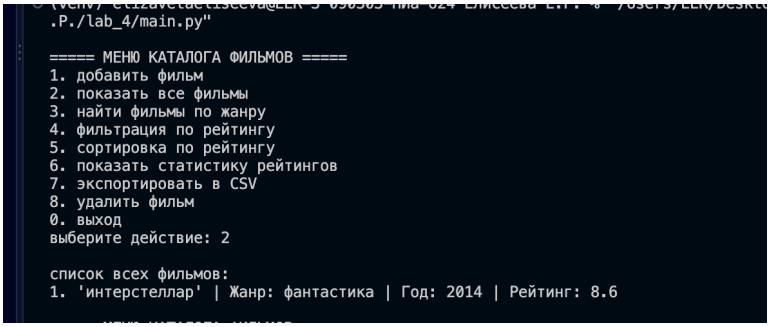


Рисунок 4.4 - список сохраненных фильмов

Тест 4. Экспорт актуальной коллекции фильмов во внешний табличный файл таблицы data.csv показан на рисунке 4.5.

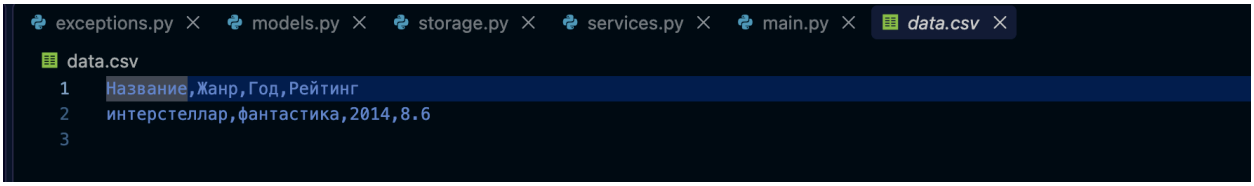


Рисунок 4.5 - табличный файл

Тест 5. Проверка работы кастомного исключения InvalidRatingError при попытке ввода некорректного значения рейтинга (например, числа -100) (см. Рисунок 4.6).

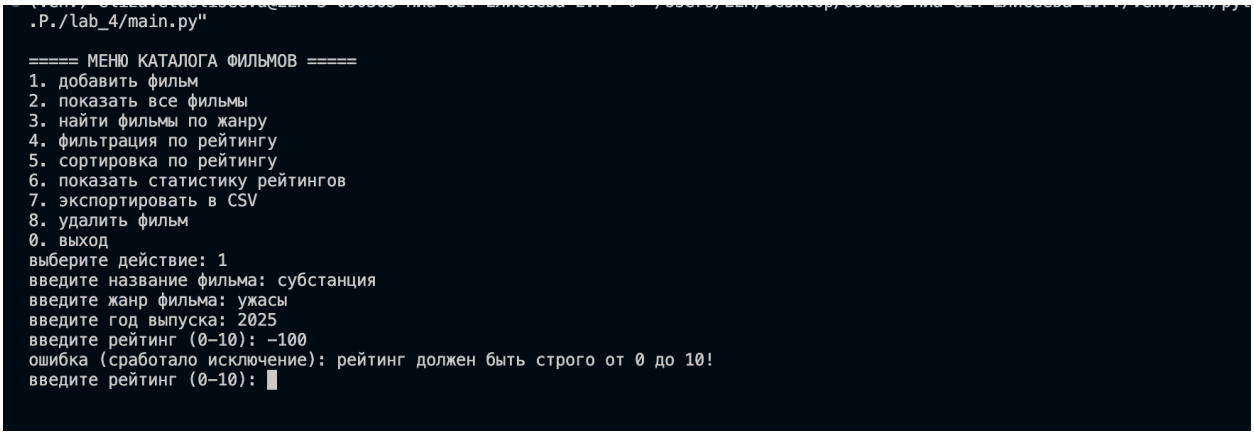


Рисунок 4.6 - некорректный рейтинг

6. Вывод

В процессе выполнения лабораторной работы были успешно освоены принципы создания многофайловых приложений на языке Python. Научились разграничивать уровни ответственности между модулями хранения данных (storage.py), описания объектов (models.py), реализации логики обработки (services.py) и интерфейса (main.py). Реализовали механизмы сериализации и десериализации объектов классов в JSON формат с применением декоратора @classmethod. Были практически освоены навыки работы с табличными

файлами (CSV), а также обработка стандартных ошибок и создание кастомных исключений для повышения общей отказоустойчивости программы к неверным действиям пользователя.